

Phase 2 (Defect B) — Cloudflare-Gated Curated Provider Design

Field	Value
Date	2026-06-13
Author	research+design subagent
Status	DESIGN — research/evidence + proposed structure; NO code written
Scope	Add a Cloudflare-gated curated provider (1337x canonical) routed through a FlareSolvrr seam
Classification	project-specific (Lava curated-provider feature; the FlareSolvrr-client pattern is reusable but the 1337x parser is Lava-domain)
Constraint compliance	No code edited; no <code>curated.go</code> / <code>api-source.hash</code> touched; no push; bounded curls only

§11.4.6 **no-guessing**: every load-bearing claim below is backed by a captured command output OR explicitly marked UNCONFIRMED: with what is needed to confirm it. The 1337x results-table selectors are UNCONFIRMED: because the page is unreachable without FlareSolvrr (proven below).

0. Verdict (read first)

BLOCKED — not ready to implement as-is.

The single most important seam fact: **the existing FlareSolvrr integration is entirely Jackett-internal — there is NO Go code in `lava-api-go` that talks to FlareSolvrr.** `lava-api-go` talks **Torznab to Jackett** (`internal/jackett/client.go`); **Jackett** holds the FlareSolvrr URL in its own config and calls FlareSolvrr's `/v1` API behind the scenes. The "seam" a curated CF provider needs — an **arbitrary-URL HTML fetch through FlareSolvrr from Go** — **does not exist yet and must be built.**

What blocks implementation:

1. **No Go FlareSolvrr client.** A new `internal/provider/flaresolvrr` package (or similar) must be written: `POST /v1 {"cmd":"request.get","url":...}`, parse the `solution.response` HTML. This is net-new Go code, not a generalization of an existing function.
2. **1337x results-table selectors are UNCONFIRMED.** The real results page is CF-gated (proven §2) — I cannot fetch it without a running FlareSolvrr to read the live HTML and pin the goquery selectors. Selectors in §3 are derived from 1337x's well-known historical structure and marked UNCONFIRMED:.
3. **Runtime dependency on a sidecar contradicts the curated-provider thesis.** Phase-1 curated providers are "zero external dependency, compiled into `liblavaapi.so`" (`curated.go` package doc). A CF-gated provider needs a live FlareSolvrr container (heavy headless Chromium). That is an **architectural decision for the operator**, not a mechanical add — see §4.

If the operator accepts the FlareSolvrr runtime dependency for curated providers, the implementation path is clear and is specified in §3 + §5.

1. The existing FlareSolvrr seam — precise map

1.1 What actually exists (evidence)

`grep` for `flaresolvrr` across `lava-api-go/internal/` returns **zero Go source hits** — only comments in curated clients noting "no Cloudflare". The only `flaresolvrr` references in tracked code are:

- `.env.example` (image + host/port/log-level env knobs)
- `tools/lava-containers/docker-compose.jackett.yml` (the `lava-flaresolvrr` service)
- docs (`docs/guides/jackett-sidecar.md`, `docs/qa/jackett-local-stack-research.md`)

There is no `request.get` / `sessions.create` call anywhere in Go.

1.2 The Jackett seam (what lava-api-go DOES talk to)

lava-api-go/internal/jackett/client.go:

- type Config { BaseURL, APIKey string; Timeout time.Duration } — both injected at runtime (§6.R); LAVA_API_JACKETT_URL (default `http://127.0.0.1:9117`) + LAVA_API_JACKETT_APIKEY from `.env`.
- Client.Search(ctx, indexerID, query) ([]Result, error) — GET `<base>/api/v2.0/indexers/<id>/results/torznab/api?apikey=&t=search&q=`, parses Torznab XML (ParseResults).
- Client.Download(ctx, downloadURL) — resolves a Torznab item's `/dl/` link; the HTTP client uses CheckRedirect: ErrUseLastResponse to capture a 302 → Location: magnet:... instead of following it.

FlareSolverr is invisible to this code. Per docs/qa/jackett-local-stack-research.md §4 + the compose fragment comments: Jackett is pointed at `http://lava-flaresolverr:8191` **inside Jackett's own config** (POST .../indexers/<id>/config at provisioning time, or global config). When an indexer is CF-gated, **Jackett** calls FlareSolverr; lava-api-go just sees Torznab results.

1.3 The FlareSolverr /v1 API (what a NEW Go client would call)

Captured from docs/qa/jackett-local-stack-research.md:212–213 + docs/guides/jackett-sidecar.md:149 + the compose healthcheck:

- FlareSolverr is its own container on port **8191**, JSON POST API at **/v1**.
- Commands: request.get, request.post, sessions.create / sessions.list / sessions.destroy.
- Healthcheck (compose, verified working 2026-06-08): POST /v1 {"cmd":"sessions.list"} → HTTP 200. The image ships curl but **not** wget (forensic note in the compose file, line 132-134).

request.get request/response shape (FlareSolverr public API; the de-facto contract Jackett uses):

```
// POST http://<flaresolverr>:8191/v1
// Content-Type: application/json
{
  "cmd": "request.get",
  "url": "https://1337x.to/search/ubuntu/1/",
  "maxTimeout": 60000,           // ms; Chromium challenge-solve
  budget
  "session": "lava-1337x"       // optional; reuse a warm
  Chromium session
}
```

```
// 200 response (abbreviated)
{
  "status": "ok",
  "message": "Challenge solved!",
  "solution": {
    "url": "https://1337x.to/search/ubuntu/1/",
    "status": 200,
    "response": "<!DOCTYPE html> ... full solved HTML ... </html>",
    "cookies": [ { "name": "cf_clearance", ... } ],
    "userAgent": "Mozilla/5.0 ..."
  }
}
```

UNCONFIRMED: the exact JSON field names above (`solution.response`, `solution.status`) are from the FlareSolverr public API documentation, NOT captured from a live response in this repo (no FlareSolverr is running on this host). They are stable across FlareSolverr v3.x but MUST be verified against the pinned \$`{LAVA_FLARESOLVERR_IMAGE}` before the parser depends on them — capture one real `request.get` response and pin the struct to it.

1.4 The curated-provider contract (what a new provider must satisfy)

From `internal/provider/curated/thepiratebay/{client,provider}.go` + `internal/provider/provider.go`:

- Each provider is a `provider.Provider` (interface at `provider.go:244`), conventionally split into a `Client` (HTTP + parse) and a `ProviderAdapter` embedding `provider.BaseProvider`.
 - Registered with one line in `curated.go` `RegisterAll` (the §6.J registration-parity guard — one site feeds both `internal/mobile` embed and `cmd/lava-api-go`).
 - Capability constants (`provider.go:43–50`): `CapSearch = "SEARCH"`, `CapMagnetLink = "MAGNET_LINK"`. `AuthType` (`provider.go:61`): `AuthNone = "NONE"`.
 - `Search(ctx, query, page) (*provider.SearchResult, error)` returns `[]provider.SearchItem{ ID, Title, InfoHash, MagnetLink, SizeBytes, Seeders, Leechers, Date, Category }`.
 - Magnet built from `info_hash + name + publicTrackers` commons (TPB pattern).
 - Error mapping: `provider.ErrNoData`, `ErrNotFound`, `ErrForbidden`, `ErrUnknown`.
-

2. 1337x Cloudflare proof (captured 2026-06-13)

A plain GET of the search path **and** the homepage is blocked by a CF interactive challenge — proven, not assumed:

```
$ curl -sS -A "Mozilla/5.0 ... Chrome/120.0 Safari/537.36" \
    "https://1337x.to/search/ubuntu/1/"
HTTP_CODE=403
```

```
# response headers:
HTTP/2 403
cf-mitigated: challenge
server: cloudflare
cf-ray: a0aff2583c2d5ebd-OSL
```

```
# response body (5617 bytes):
<title>Just a moment...</title>
Enable JavaScript and cookies
_cf_chl          (x8)
challenge-platform
```

```
$ curl ... "https://1337x.to/"          # homepage too
HTTP/2 403
cf-mitigated: challenge
server: cloudflare
```

Conclusion (CONFIRMED): 1337x serves the Cloudflare "Just a moment..." JS/cookie interstitial (HTTP 403, cf-mitigated: challenge, challenge-platform, _cf_chl) on every plain GET. A curated provider **cannot** use a direct http.Get like the Phase-1 providers; it **MUST** route through a challenge-solver (FlareSolvrr). This is exactly the IPTorrents class .env.example:38 describes.

Note on TorrentGalaxy: per the task brief it is DNS-dead — **skip it**. 1337x is the canonical Phase-2 candidate. Not re-verified here (out of scope).

3. Proposed provider design

Two new packages (mirroring the Client / Adapter split, plus the missing seam):

```
internal/provider/flaresolvrr/          # NEW — the missing seam
(reusable)
    client.go          # FlareSolvrrClient: POST /v1 request.get →
solved HTML
internal/provider/curated/onethreethreesevenx/  # NEW curated
```

```

provider
    client.go          # Search: build URL → FlareSolvrr fetch →
goquery parse
    provider.go        # ProviderAdapter: SEARCH + MAGNET_LINK,
AuthNone, Kind native

```

(Package name onethreethreesevenx because a Go identifier cannot start with a digit; providerID = "1337x" is the stable catalogue id, set as a string constant — same split TPB uses between package name and providerID.)

3.1 The FlareSolvrr Go client (internal/provider/flaresolvrr/client.go)

```

type Config struct {
    BaseURL    string          // LAVA_API_FLARESOLVERR_URL, e.g.
                http://127.0.0.1:8191 ($6.R, NEW env var)
    Timeout    time.Duration // bounds the Go-side request;
                default 70s (> maxTimeout)
    MaxTimeout time.Duration // FlareSolvrr's own Chromium
                budget; default 60s
}

// Get solves the CF challenge for url and returns the solved HTML
// + status.
func (c *Client) Get(ctx context.Context, url string) (html
    string, status int, err error)
//    POST {BaseURL}/v1
//        {"cmd":"request.get","url":url,"maxTimeout":<ms>,"session":"lava-1337x"}
//    on resp.status=="ok" && solution.status==200 → return
//        solution.response
//    else → provider.ErrUnknown (or ErrForbidden if
//        solution.status==403)

```

- **\$6.R:** LAVA_API_FLARESOLVERR_URL is a NEW .env knob (the LAVA_FLARESOLVERR_* vars today only feed the compose container; the Go consumer URL is net-new, mirroring how LAVA_API_JACKETT_URL differs from LAVA_JACKETT_*). Default http://127.0.0.1:8191 per the loopback-publish topology.
- **\$6.B:** an optional Health(ctx) posting {"cmd":"sessions.list"} (the same probe the compose healthcheck uses) so the provider's HealthCheck honestly reports Healthy:false when FlareSolvrr is down.
- **\$6.AC:** on any solve failure, observability.RecordNonFatal(ctx, err, attrs) with module="flaresolvrr", operation="request.get", error_class, redacted url.

3.2 The 1337x provider client (.../onethreethreesevenx/client.go)

```
const DefaultBaseURL = "https://1337x.to" // §6.R: this
      provider's fixed upstream identity
const searchPathFmt = "/search/%s/%d/" // /search/<query>/
      <page>/

func (c *Client) Search(ctx, query string, page int)
    (*provider.SearchResult, error) {
    if page < 1 { page = 1 }
    u := c.baseURL + fmt.Sprintf(searchPathFmt,
        url.PathEscape(query), page)
    html, status, err := c.flare.Get(ctx, u) // <-- the seam:
        FlareSolvErr, NOT http.Get
    // map status → provider.Err*
    items := parseSearchHTML(html) // goquery
    // 1337x search rows DO NOT carry the magnet – only a detail-
        page link +
    // seeders/leechers/size. Two-step is required (see §3.4).
}
```

3.3 goquery selectors — UNCONFIRMED:

UNCONFIRMED: The real solved HTML is unreachable from this host (no FlareSolvErr running — §2 proves the page is CF-gated). The selectors below are 1337x's well-known historical results-table structure; they **MUST** be pinned against a captured solved page before the parser is trusted. **To confirm:** bring up the cloudflare compose profile, run one request.get for /search/ubuntu/1/, save solution.response to testdata/, and verify each selector against it.

Search results table (UNCONFIRMED:):

Field	Selector (relative to table.table-list tbody tr)	Notes
Row	table.table-list tbody tr	one torrent per row
Title + detail link	td.coll-1.name a:nth-of-type(2)	href="/torrent/<id>/<slug>/" ; text = title. (first a is a category icon)
Seeders	td.coll-2.seeds	integer text
Leechers	td.coll-3.leeches	integer text
Date added	td.coll-date	display date
Size	td.coll-4.size	

Field	Selector (relative to <code>table.table-list tbody tr</code>)	Notes
		e.g. "3.2 GB"; contains a nested <code> seeders-dup</code> — take the direct text node

Detail page (UNCONFIRMED:) — for the magnet (search rows lack it):

Field	Selector	Notes
Magnet	<code>a[href^="magnet:"]</code>	the magnet URI is on the torrent detail page, not the results row
Info hash	<code>parse xt=urn:btih:<hash> from the magnet</code>	OR <code>div.infohash-box span</code>

3.4 Capability honesty (§6.E) — important caveat

1337x search rows do **not** embed the magnet (unlike apibay/TPB). The magnet is on the **detail page**. Therefore an honest 1337x provider has two options:

- **(A) Two-step in Search:** for each results row, do a second FlareSolvrr request.get on the detail page to extract the magnet. **Cost:** N+1 headless- Chromium solves per search — very heavy (FlareSolvrr runs Chromium per request). Likely too slow for a live search.
- **(B) Defer the magnet to GetTopic/GetTorrent:** Search returns rows with ID = detailPath, Title, Seeders, Leechers, Size, **but no MagnetLink**; the magnet is resolved lazily when the user taps a result (GetTopic). This means the provider declares CapSearch + CapTopic + CapMagnetLink and Search items carry an empty MagnetLink until topic resolution.

§6.E capability honesty: Option B is the correct anti-bluff shape — declaring CapMagnetLink while Search returns empty magnets would be dishonest unless the capability is understood as "magnet available via topic resolution". The design recommends **Option B:**

Capabilities() = [CapSearch, CapTopic, CapMagnetLink],
GetTopic does the single detail-page FlareSolvrr solve and returns the magnet. This keeps a search to ONE solve and defers the second solve to an explicit user tap. The `_ provider.ErrUnsupported` stubs for GetTopic/GetTorrent that TPB uses are REPLACED with a real GetTopic here.

3.5 provider.go adapter (mirrors TPB, deltas only)

- `providerID = "1337x", DisplayName() = "1337x", canonicalSite = "https://1337x.to".`
 - `AuthType() = provider.AuthNone, SupportsAnonymous() = true, Kind() inherits "native".`
 - `Capabilities() = [CapSearch, CapTopic, CapMagnetLink]` (per §3.4 Option B).
 - Search delegates to `client.Search`; `GetTopic` delegates to `client.GetTopic` (detail-page solve → magnet). All other methods return `provider.ErrUnsupported` (TPB pattern).
 - `HealthCheck` → `client.Health` → `flare.Health` (`FlareSolvErr` sessions.list).
 - Registration: **one** line added to `curated.go` `RegisterAll` (`r.Register(oneethreethreesevenx.New(flareClient))`). **Out of scope for this doc — do NOT touch curated.go per the constraint; this is the implementation step.**
-

4. Seam-generalization assessment

Q: Does the existing seam support an arbitrary-URL fetch (what a curated CF provider needs), or is it Jackett-specific?

A: It is 100% Jackett-specific. There is NO reusable arbitrary-URL FlareSolvErr fetch in Go at all. Specifically:

1. `internal/jackett/client.go` only knows Torznab — it builds `/api/v2.0/indexers/.../torznab/api` URLs and GETs Jackett. It cannot fetch an arbitrary URL, and it never touches FlareSolvErr.
2. FlareSolvErr is reached **only by Jackett's own .NET process**, configured via Jackett's indexer/global config (`http://lava-flaresolvErr:8191`). No Go code posts to `/v1`.
3. The compose fragment brings FlareSolvErr up under the `cloudflare` profile, but nothing in `lava-api-go` consumes it directly.

Minimal generalization needed (the net-new work):

- **Write `internal/provider/flaresolvErr/client.go`** — a small, reusable Go client: `Get(ctx, url) → (html, status, err)` over `POST /v1 request.get`, plus `Health` over `sessions.list`. ~80-120 lines. This IS the seam; it is the prerequisite for ANY curated CF provider (1337x, and later any other CF tracker).
- **Add `LAVA_API_FLARESOLVERR_URL` to config** — `internal/config/config.go` + `.env.example` (the Go consumer URL; distinct from the

LAVA_FLARESOLVERR_* container knobs, exactly as
LAVA_API_JACKETT_URL is distinct from LAVA_JACKETT_*).

- **Decide the runtime-dependency posture (operator call).** Phase-1 curated providers are explicitly "zero external dependency, compiled into liblavaapi.so" (curated.go doc). A CF-gated curated provider breaks that invariant — it needs a live FlareSolvrr sidecar (heavy Chromium). Options:
 - **Gate it behind availability:** register the 1337x provider only when LAVA_API_FLARESOLVERR_URL is set + Health passes; otherwise it is absent from the catalogue (honest — §6.E: no provider listed that cannot complete its flow). This is the recommended posture.
 - This is a **design decision**, not a mechanical add — it determines whether 1337x ships in the always-on curated set or as an opt-in CF set.

5. Ready-to-implement checklist

#	Item	Status
1	Prove 1337x needs FlareSolvrr	✅ CONFIRMED (§2: 403 + cf-mitigated + Just-a-moment)
2	Map the existing seam	✅ CONFIRMED (§1: seam is Jackett-internal; no Go FlareSolvrr client exists)
3	FlareSolvrr /v1 request.get response field names	⚠️ UNCONFIRMED: — pin against a captured live response from \$ {LAVA_FLARESOLVERR_IMAGE}
4	1337x results-table goquery selectors	⚠️ UNCONFIRMED: — pin against a captured solved page (needs FlareSolvrr up)
5	New internal/provider/flaresolvrr Go client	❌ Net-new code (the missing seam)
6	LAVA_API_FLARESOLVERR_URL config + .env.example	❌ Net-new env knob
7	Operator decision: runtime FlareSolvrr dependency for curated set	❌ BLOCKING design decision (§4)
8	Magnet via two-step (§3.4 Option B: Search + GetTopic)	⚠️ design recommends B; implementation choice

Verdict: BLOCKED. Three things gate implementation: (a) the FlareSolvrr Go client + config knob do not exist and must be built; (b) two selector/field sets are UNCONFIRMED: and need a live FlareSolvrr to pin; (c) the operator

must accept (or scope) the FlareSolvrr runtime dependency for the curated set. Once the operator green-lights (c) and a FlareSolvrr container is up to confirm (3) and (4), the implementation path in §3 is concrete and follows the established TPB Client/Adapter pattern with one real delta (the FlareSolvrr seam + two-step magnet).

Anti-bluff falsifiability note (per §6.J / §6.L)

This is a design doc, not code, so no test mutation applies. The honesty load-bearing claims are: the §2 CF proof is a real captured curl (not assumed), and the §4 "no Go FlareSolvrr client exists" claim is a real grep result (zero source hits). Both are reproducible: re-run the §2 curl and re-run

```
grep -rn flaresolvrr lava-api-go/internal/ --include='*.go' |  
grep -v _test.
```